

Sprint 5 - Software Engineering Essentials, Java & OOAD

Answer Sheet | 20 Questions | Correct answers marked with a check

Q1

Select ONE answer

You're reviewing a trade-allocation utility that is being shared between a CSV import job and a later HTTP service. One helper class exposes its running quantity as a public field, and two different callers are already mutating it in inconsistent ways during a reconciliation window. You refactor the class so the field becomes private, callers can only read it through an accessor, and updates must go through a method that rejects any change that would make the quantity negative. What OO principle have you primarily applied?

A. Polymorphism, because the class can now behave differently depending on the type of object it receives from the caller

✓ **B. Encapsulation - by making the field private, exposing a read-only accessor, and routing all mutations through a method.**

C. Interface segregation, because the class exposes a smaller, more focused public API instead of a broad set of mutators

D. Dependency inversion, because the value update logic no longer depends directly on its callers and can be reused from any context

Q2

Select ALL that apply

You're translating your Python instincts into Java for the first time this sprint. Which of the following statements about the Python-to-Java transition are accurate? (Select all that apply.)

✓ **A. Looking for a Java concept's Python equivalent first is usually a productive way to reason about unfamiliar Java syntax**

✓ **B. Most core ideas - types, collections, exceptions, control flow - transfer directly between the two languages; it's mainly the syntax and Java's compile-time strictness that differ**

C. Java has no equivalent of Python's exception handling, so all error handling must use return codes instead

✓ **D. Java's static typing means a type mismatch that Python would only surface at runtime (or not at all) can be caught by the compiler before the code ever runs**

E. Because Python is dynamically typed, any valid Python program can be mechanically translated into Java without redesigning any types

Q3*Select ONE answer*

You're in an OOAD workshop for a trading system, turning a requirements sheet into a first-draft object model. One person wants RiskLimit as a dedicated class because it has its own validation rules and reporting needs; another wants it as a plain attribute on Client because that keeps the model simpler. Your colleague insists there must be one objectively correct answer. How should you respond?

- A. Agree - one of you must be making an analysis error, so keep debating until you converge
- B. Refuse to proceed until the answer is looked up in a UML reference

✓ **C. Point out that this kind of class-vs-attribute decision is often genuinely ambiguous - write down both views and the reasoning, since that disagreement is useful input for later review and refinement**

- D. Flip a coin, since the choice has no real consequences for the resulting design

Q4*Select ONE answer*

During a design review for an order-processing feature, you notice that almost every verb in the requirements document - validate, calculate, execute, produce, record - has been pushed onto one central OrderManager class, while the other candidate classes have been left as little more than data holders. The team says this is fine because it keeps the workflow in one place. What should this pattern make you suspicious of?

- A. That OrderManager is missing a no-argument constructor

✓ **B. A "God class" design smell, where responsibilities that belong on other objects have been dumped onto one manager class - a problem SOLID (particularly SRP) will formally address later**

- C. That the requirements document itself is incomplete
- D. That the team needs more classes with private fields

Q5*Select ALL that apply*

You're drawing a UML class diagram in Mermaid from an OOAD worksheet. For each "has a" relationship you identified on paper, you need to choose the correct notation. Which of the following pairings are correct? (Select all that apply.)

- ✓ **A. "Portfolio references Instruments that continue to exist in the market independently of the Portfolio" → aggregation, Portfolio o-- Instrument**
- ✓ **B. "Portfolio has many Holdings, and a Holding is meaningless without its Portfolio" → composition, Portfolio *-- Holding**
- ✓ **C. "CsvSettlementReport implements the CsvReportable interface" → realization, CsvSettlementReport ..|> CsvReportable**

D. "OrderValidator uses a RiskLimit object only as a method parameter, and doesn't hold onto it afterward" → association, OrderValidator ..> RiskLimit

E. "BondInstrument is a kind of Instrument" → association, BondInstrument --> Instrument

Q6*Select ONE answer*

You're building a UML sequence diagram for an "order submission" workflow with participants Client, OrderValidator, and OrderExecutor. You want to show that execution only happens for valid orders, and that rejected orders are reported with a reason instead. What is the correct way to model this?

- A. Draw two entirely separate sequence diagrams, one for valid and one for invalid, with no way to see they're alternatives
- ✓ **B. Wrap the diverging behaviour in a single alt fragment, with one branch showing execution/acceptance and the other showing rejection with a reason**
- C. Add a loop fragment around the validation call so it retries until the order becomes valid
- D. Omit the invalid case entirely, since sequence diagrams can only show one path through a workflow

Q7*Select ALL that apply*

You're reviewing a partner's design (class diagram + sequence diagram) and are receiving their review of yours in return. Which of the following reflect good peer review practice? (Select all that apply.)

- ✓ **A. If you disagree with a comment on your own design, replying with your reasoning and checking whether it addresses the reviewer's concern, rather than dismissing it outright**
- B. Only leaving comments verbally, so there's no written record that could feel confrontational
- ✓ **C. Leaving at least two specific, constructive, written comments rather than a vague "looks fine"**

D. Rewriting your partner's diagram yourself and handing back the corrected version instead of comments

Q8*Select ALL that apply*

Your team's codebase has a class `FrozenHoldingBad` that extends `Holding` and overrides `adjust()` to always throw an exception, so that any code calling `adjust()` on it (as it could on any other `Holding`) breaks unexpectedly. Which of the following statements about this situation are correct? (Select all that apply.)

A. LSP violations are only a style preference and have no effect on client code correctness

✓ **B. This is a Liskov Substitution Principle violation, because `FrozenHoldingBad` cannot be substituted everywhere a `Holding` is expected without changing correct behaviour**

C. The correct fix is to make `Holding`'s `adjust()` method final so it can never be overridden

✓ **D. A better fix is for `FrozenHolding` to hold a `Holding` as a field (composition) rather than extend it, and simply not expose an `adjust()` method at all**

Q9*Select ALL that apply*

Your team needs to add a new `DerivativeInstrument` type (a flat 2% fee) to an existing system of `BondInstrument`, `EquityInstrument`, and `FundInstrument`, all implementing a common `Feeable` interface via an abstract `Instrument` class. The acceptance criteria say this must be done without modifying `Instrument.java`, `Feeable.java`, `Order.java`, or any existing instrument class. Which statements correctly describe why this matters? (Select all that apply.)

✓ **A. Extending `Instrument` for the new type keeps the polymorphic contract intact, so existing code that works with `Instrument` or `Feeable` references continues to work unchanged**

✓ **B. Not touching existing classes means their existing test coverage remains valid evidence that they still work correctly**

C. It's required because Java does not allow modifying a class once any subclass of it exists

✓ **D. This is an application of the Open/Closed Principle - the system should be open for extension (new instrument types) but closed for modification (existing, already-tested classes stay untouched)**

E. This is a Liskov Substitution Principle concern, not an Open/Closed Principle concern

Q10*Select ONE answer*

A legacy report builder in a fee analytics module originally calculated totals, formatted the report body, and handled file output in one method. You split it into `FeeAggregator`, which only sums fees, and `FeeReportFormatter`, which only assembles the output string for downstream delivery. Neither class knows anything about the other's job anymore. Which SOLID principle does this directly demonstrate?

A. Open/Closed Principle

✓ **B. Single Responsibility Principle**

C. Interface Segregation Principle

D. Dependency Inversion Principle

Q11*Select ALL that apply*

A colleague's `OrderExecutor` class currently does this: `"new ConsoleReportWriter()"` is called directly inside `execute()`, so every order writes straight to the console with no way to redirect or test that output. You suggest changing the constructor to accept a `ReportWriter` interface parameter instead, and store that. Which of the following are true about this change? (Select all that apply.)

A. It eliminates the need for any interface at all, since `ReportWriter` can now be a concrete class

B. It is primarily an Interface Segregation Principle change, since it reduces the number of methods on `ReportWriter`

✓ **C. It becomes possible to substitute an `InMemoryReportWriter` in tests, so behaviour can be verified without depending on console output**

✓ **D. It applies the Dependency Inversion Principle: the high-level `OrderExecutor` now depends on an abstraction (`ReportWriter`) rather than constructing a concrete, low-level implementation itself**

✓ **E. `OrderExecutor` no longer needs to change if a new kind of `ReportWriter` (e.g. writing to a file) is introduced later**

Q12*Select ALL that apply*

Your team has a fat `CsvReportable/ConsoleReportable`-style interface design question: a new `CsvSettlementReport` class only ever needs to produce CSV output, never console output. A colleague suggests implementing both `CsvReportable` and `ConsoleReportable` "just in case it's needed later." Which statements correctly explain why this is worth pushing back on in review? (Select all that apply.)

A. `ConsoleReportable` and `CsvReportable` cannot logically coexist on the same object under any circumstances, in any design

✓ **B. "Just in case it's needed later" adds coupling and complexity for a requirement that doesn't currently exist, rather than keeping the class focused on what it actually does**

✓ **C. It violates the Interface Segregation Principle - `CsvSettlementReport` would be forced to depend on methods (console formatting) it doesn't need or use**

✓ **D. A test that verifies `CsvSettlementReport`'s class hierarchy (e.g. via reflection) confirming it implements only `CsvReportable` would fail if `ConsoleReportable` were added unnecessarily**

E. Java classes are not permitted to implement more than one interface, so the suggestion wouldn't even compile

Q13*Select ONE answer*

A colleague pastes a messy `OrderCategoriser` class into GitHub Copilot Chat and gets back a polished-looking refactor that renames a few variables, extracts two helper methods, and changes the structure enough to look "cleaner" at a glance. The code compiles, but no one has yet compared the behaviour against the original version. Following this sprint's guidance on critically assessing GenAI suggestions, what should the colleague do before accepting it?

A. Accept it immediately, since AI-suggested refactors are always behaviour-preserving by construction

✓ **B. Run the existing, unmodified test suite against the refactored version and evaluate each specific change against a checklist (e.g. does it actually improve readability, does it preserve behaviour), rather than accepting confident-looking code uncritically**

C. Reject it outright without reading it, since GenAI suggestions are never useful for refactoring

D. Modify the existing test suite to match whatever the suggested refactor produces

Q14*Select ONE answer*

You're building a `FeeBandClassifier` under strict TDD for a pricing service that will later be extended with additional bands. For the very first red-green cycle, the only test says `classify(1000)` should return "STANDARD". A colleague reviewing your commit is surprised that your green implementation just returns "STANDARD" unconditionally, with no conditional logic at all. Is this a shortcut you're getting away with?

A. Yes - you should always implement the full business rule from the spec, even for the first test

✓ **B. No - writing the smallest implementation that makes the current failing test pass is the correct TDD discipline; the next test is what will force real conditional logic to be added**

C. Yes - TDD requires writing all tests up front before any implementation code

D. No, but only because "STANDARD" happens to be the most common band in production data

Q15*Select ALL that apply*

You're writing a JUnit 5 test suite for `RiskLimitChecker.canAcceptOrder(currentPortfolioValue, orderValue, riskLimit)`, which returns true/false based on whether the order fits within the risk limit, and throws `IllegalArgumentException` for `orderValue <= 0` or `riskLimit < 0`. Which of the following are genuinely good uses of JUnit 5 features for this class, as opposed to using `@Test` and `assertEquals` for everything? (Select all that apply.)

✓ **A. Using `assertThrows` to verify `IllegalArgumentException` is thrown for `orderValue <= 0`, and a separate test for `riskLimit < 0`**

B. Using `@Mock` and Mockito's `when(...).thenReturn(...)` on `RiskLimitChecker` itself, since it takes only primitive parameters and has no collaborators to isolate

C. Avoiding `@BeforeEach` and instead constructing a new `RiskLimitChecker` inline in every single test method, since setup logic should never be shared

✓ **D. Using `@Nested` classes (e.g. "WhenOrderIsValid" and "WhenInputIsValid") with `@DisplayName` annotations so the test report documents the behaviour being tested**

✓ **E. Using `@ParameterizedTest` with `@CsvSource` to cover several (`currentPortfolioValue`, `orderValue`, `riskLimit`) combinations, including the exact-boundary case where `currentPortfolioValue + orderValue == riskLimit`**

Q16*Select ALL that apply*

A colleague's class, `OrderExecutor`, depends on both an `Instrument` and a `ReportWriter`. In a JUnit test, they use real `BondInstrument` and `InMemoryReportWriter` objects rather than mocks. A teammate suggests rewriting the test with Mockito's `@Mock` and `verify(...)` instead. Which statements correctly describe the trade-off being weighed? (Select all that apply.)

A. Mockito tests always run faster than tests using real collaborators, which is the only consideration that matters

✓ **B. Using real collaborators exercises more of the actual integrated behaviour, but a red result could originate anywhere in the chain, not just in `OrderExecutor`**

C. Mocking is required whenever a class has more than one constructor parameter, regardless of context

✓ **D. Using mocks isolates the test so a red result means `OrderExecutor` itself is broken, not one of its collaborators**

✓ **E. Not every class needs a mock - a class like `RiskLimitChecker` that takes only primitive parameters has no collaborators worth isolating in the first place**

Q17*Select ONE answer*

While wiring together already-tested classes in `OrderProcessingEngine`, you notice a tempting shortcut: instead of calling `Instrument.calculateFee()`, you could copy the fee calculation into the orchestrator and tweak it for one edge case that the finance team keeps asking about. The team argues that it would be "quicker" and would avoid one method call. What should this urge signal to you?

A. That `OrderProcessingEngine`'s constructor needs an additional parameter

✓ **B. That you're at risk of duplicating logic that already has a clear owner - the orchestrator's job is coordination only, not re-deciding fee, validation, or adjustment rules that earlier classes already own and have tested**

C. That `Instrument.calculateFee()` must contain a bug, since otherwise there'd be no need to reconsider it

D. That the mission build module doesn't need any tests, since the components were already tested individually

Q18*Select ONE answer*

In the order-processing flow, `OrderProcessingEngine.process()` looks up a `Client`, validates a proposed order via `OrderValidator`, and - only if valid - calculates a fee, updates the `Holding`, and adjusts the `Portfolio`'s total value. In a design review, someone asks what should happen when the order is rejected after validation fails. What should the correct behaviour be, per the mission's requirements?

A. The portfolio value should be adjusted anyway, since the order was still attempted

✓ **B. The order is recorded as rejected with a reason, and everything else about the client's portfolio and holdings is left completely unchanged**

C. The holding should be adjusted but the portfolio value left unchanged

D. The client's entire portfolio should be reset to its last known-good state

Q19*Select ALL that apply*

Comparing a first-draft object model against the class actually built, a colleague finds that "RiskLimit" ended up as a plain attribute on `Client` rather than its own class, contradicting their original worksheet entry. Which of these are reasonable, legitimate causes for a design decision changing between an early OOAD draft and the final build? (Select all that apply.)

✓ **A. A SOLID violation (e.g. an accidental Single Responsibility Principle breach) caught your attention forced a rethink of where a responsibility should live**

B. Design decisions made early in the process are contractually final and cannot legitimately change afterward

✓ **C. A peer-review comment surfaced a concern with the original design that prompted reconsideration**

D. Any change between draft design and final build indicates the original OOAD process was worthless

✓ **E. Writing an actual failing test in TDD revealed that the original class boundary didn't match how the behaviour was actually needed**

Q20*Select ONE answer*

You're building a portfolio analytics service and need to choose the right Java collection for three separate requirements: (1) maintain an ordered history of trade events that may include duplicates, (2) track which instrument IDs have already been processed in a reconciliation run, ensuring no duplicates without caring about order, and (3) look up the current holding quantity by instrument symbol in $O(1)$ time. Which combination of Java collection types best satisfies all three requirements?

A. List for (1), List for (2), and List for (3) - a List can handle all three cases if you write the right loops

B. Set for (1), Map for (2), and List for (3) - Sets preserve insertion order and Maps can store duplicates

✓ **C. List for (1), Set for (2), and Map for (3) - a List preserves order and allows duplicates, a Set eliminates duplicates, and a Map provides key-based lookup**

D. Map for (1), List for (2), and Set for (3) - Maps maintain insertion order, Lists deduplicate, and Sets support key-based lookup